EECE 7205: Fundamentals of Computer Engineering

Fall 2026

Assignment: Binary Tree Questions

Student Name: Natan Blumberg
Instructor: Dr. Fatema Nafa

Question 1 (5 points): Maximum Depth of a Binary Tree

(a) Pseudocode

The depth of a tree is one more than the larger depth of its two subtrees. An empty tree has depth -1, so a leaf gets $1 + \max(-1, -1) = 0$.

Algorithm 1: Maximum Depth
Input: a pointer `node` to the root of a binary tree (NULL for an empty tree).
Output: the number of edges on the longest root-to-leaf path (-1 for an empty tree).
Initial call: `MAXDEPTH(root)`

```
1: procedure MAXDEPTH(node)
2:     if node = NULL then
3:         return -1
4:     end if
5:     leftDepth <- MAXDEPTH(node.left)
6:     rightDepth <- MAXDEPTH(node.right)
7:     return 1 + MAX(leftDepth, rightDepth)
8: end procedure
```

(b) Worked Example

Calls are listed in the order they start. Each call returns only after all calls below it have returned.

#	Call	Lines executed	Returns
1	MAXDEPTH(5)	2, 5, 6, 7	2
2	MAXDEPTH(12)	2, 5, 6, 7	1
3	MAXDEPTH(21)	2, 5, 6, 7	0
4	MAXDEPTH(NULL), left of 21	2, 3	-1
5	MAXDEPTH(NULL), right of 21	2, 3	-1
6	MAXDEPTH(3)	2, 5, 6, 7	0
7	MAXDEPTH(NULL), left of 3	2, 3	-1
8	MAXDEPTH(NULL), right of 3	2, 3	-1
9	MAXDEPTH(8)	2, 5, 6, 7	1
10	MAXDEPTH(17)	2, 5, 6, 7	0
11	MAXDEPTH(NULL), left of 17	2, 3	-1
12	MAXDEPTH(NULL), right of 17	2, 3	-1
13	MAXDEPTH(9)	2, 5, 6, 7	0
14	MAXDEPTH(NULL), left of 9	2, 3	-1
15	MAXDEPTH(NULL), right of 9	2, 3	-1

Final output: MAXDEPTH(5) returns $1 + \max(1, 1) = 2$, which matches the expected output 2.

(c) Frequency Count

Line	Operation	Cost	Frequency	Count for the example
2	Test node = NULL	1	$2n+1$	15
3	Return -1 (NULL child)	1	$n+1$	8
5	Recursive call on left child	1	n	7
6	Recursive call on right child	1	n	7
7	Return 1 + MAX	1	n	7
	Total $T(n)$		$6n+2$	44

$T(n) = (2n+1) + (n+1) + 3n = 6n+2$. Check: $T(7) = 6(7)+2 = 44$, the same as the sum of the Count column (15 + 8 + 7 + 7 + 7 = 44).

(d) Final Complexity

The dominant term of $T(n) = 6n+2$ is $6n$. Best case $\Theta(n)$, worst case $\Theta(n)$. Extra space is the call stack, $\Theta(h)$: $\Theta(\log n)$ for a complete tree and $\Theta(n)$ for a chain. Every input makes one call per node and one per NULL child, so no input is slower than another; the worst case for space is a chain, where all n calls are open at the same time.

Twist

No. A chain of 7 nodes still has 7 nodes and $n+1 = 8$ NULL child pointers, so MAXDEPTH is still called 15 times and every line has the same frequency: $T(7) = 44$ again. The count depends only on n, not on the shape of the tree.

Question 2 (5 points): Sum of All Nodes

(a) Pseudocode

Version 1 adds the node to the sums of its two subtrees. Version 2 visits the nodes in level order with a queue and adds each one to a running total.

Algorithm 2a: Sum of All Nodes (recursive)
Input: a pointer `node` to the root of a binary tree.
Output: the sum of all values in the tree (0 for an empty tree).
Initial call: `SUMTREE(root)`

```
 1: procedure SUMTREE(node)
 2:     if node = NULL then
 3:         return 0
 4:     end if
 5:     leftSum <- SUMTREE(node.left)
 6:     rightSum <- SUMTREE(node.right)
 7:     return node.data + leftSum + rightSum
 8: end procedure
```

Algorithm 2b: Sum of All Nodes (queue, level order)
Input: a pointer `root` to the root of a binary tree.
Output: the sum of all values in the tree (0 for an empty tree).
Initial call: `SUMLEVEL(root)`

```
 1: procedure SUMLEVEL(root)
 2:     if root = NULL then
 3:         return 0
 4:     end if
 5:     total <- 0
 6:     Q <- empty queue
 7:     ENQUEUE(Q, root)
 8:     while Q is not empty do
 9:         node <- DEQUEUE(Q)
10:         total <- total + node.data
11:         if node.left is not NULL then
12:             ENQUEUE(Q, node.left)
13:         end if
14:         if node.right is not NULL then
15:             ENQUEUE(Q, node.right)
16:         end if
17:     end while
18:     return total
19: end procedure
```

(b) Worked Example

Version 1 (recursive).

#	Call	Lines executed	Returns
1	SUMTREE(5)	2, 5, 6, 7	75
2	SUMTREE(12)	2, 5, 6, 7	36
3	SUMTREE(21)	2, 5, 6, 7	21
4	SUMTREE(NULL), left of 21	2, 3	0
5	SUMTREE(NULL), right of 21	2, 3	0
6	SUMTREE(3)	2, 5, 6, 7	3
7	SUMTREE(NULL), left of 3	2, 3	0
8	SUMTREE(NULL), right of 3	2, 3	0
9	SUMTREE(8)	2, 5, 6, 7	34
10	SUMTREE(17)	2, 5, 6, 7	17
11	SUMTREE(NULL), left of 17	2, 3	0
12	SUMTREE(NULL), right of 17	2, 3	0
13	SUMTREE(9)	2, 5, 6, 7	9
14	SUMTREE(NULL), left of 9	2, 3	0
15	SUMTREE(NULL), right of 9	2, 3	0

Version 2 (queue). One row per pass of the while loop.

Pass	Dequeue	total	Enqueue	Queue after (front first)
0 (start)	-	0	5	5
1	5	5	12, 8	12, 8
2	12	17	21, 3	8, 21, 3
3	8	25	17, 9	21, 3, 17, 9
4	21	46	none	3, 17, 9
5	3	49	none	17, 9
6	17	66	none	9
7	9	75	none	empty

Final output: both versions return 75 ($36 + 34 + 5$ in version 1, the running total after pass 7 in version 2), which matches the expected output.

(c) Frequency Count

Version 1 (recursive).

Line	Operation	Cost	Frequency	Count for the example
2	Test node = NULL	1	$2n + 1$	15
3	Return 0 (NULL child)	1	$n + 1$	8
5	Recursive call on left child	1	n	7
6	Recursive call on right child	1	n	7
7	Return node.data + sums	1	n	7
	Total $T(n)$		$6n + 2$	44

Check: $T(7) = 6(7) + 2 = 44$, the same as the Count column.

Version 2 (queue). Every node except the root is enqueued once, so lines 12 and 15 add to $n - 1$. For a tree built from an array, the nodes with a left child are indices 0 to $\lfloor n/2 \rfloor - 1$, which gives the two floor terms.

Line	Operation	Cost	Frequency	Count for the example
2	Test root = NULL	1	1	1
3	Return 0 (empty tree)	1	0	0
5	total $\leftarrow 0$	1	1	1
6	Create empty queue	1	1	1
7	Enqueue root	1	1	1
8	Test Q is not empty	1	$n+1$	8
9	Dequeue node	1	n	7
10	Add node.data to total	1	n	7
11	Test node.left is not NULL	1	n	7
12	Enqueue left child	1	$\lfloor n/2 \rfloor$	3
14	Test node.right is not NULL	1	n	7
15	Enqueue right child	1	$\lfloor (n-1)/2 \rfloor$	3
18	Return total	1	1	1
	Total $T(n)$		$6n+5$	47

$T(n) = 5 + (n+1) + 4n + (n-1) = 6n + 5$. Check: $T(7) = 6(7) + 5 = 47$, the same as the Count column.

(d) Final Complexity

Both counts are dominated by $6n$. For both versions: best case $\Theta(n)$ and worst case $\Theta(n)$, because every node is visited exactly once whatever the shape, so no input is worse than another. Extra space: $\Theta(h)$ for version 1 (call stack) and $\Theta(w)$ for version 2 (queue), where w is the largest number of nodes on one level; both are $\Theta(n)$ in the worst case (see the Twist).

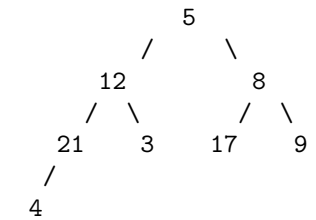
Twist

The two frequency counts are almost the same: $6n+2 = 44$ for the recursive version and $6n+5 = 47$ for the queue version. The recursive version spends its steps on the $n+1$ NULL calls; the queue version avoids NULL calls by testing each child before enqueueing it, but pays a few constant setup steps (total, queue, first enqueue, final return). The extra space is the real difference. On the reference tree the stack holds at most 4 calls (5, 12, 21, NULL) and the queue at most 4 nodes (21, 3, 17, 9). For larger trees the two behave in opposite ways: a complete tree needs only $\Theta(\log n)$ stack but $\Theta(n)$ queue, and a chain needs $\Theta(n)$ stack but only $\Theta(1)$ queue.

Question 3 (5 points): Count the Nodes That Have Exactly One Child

(a) Pseudocode

Input tree: INSERT(root, 4) uses shortcut I6: the new node goes to index $n = 7$, whose parent is index $(7 - 1)/2 = 3$, which holds 21. Index 7 is odd, so 4 becomes the left child of 21 (rule I3). The tree now has $n = 8$ nodes:



The algorithm counts the one-child nodes in both subtrees and adds 1 if the current node has a child on only one side.

Algorithm 3: Count Nodes With Exactly One Child
Input: a pointer `node` to the root of a binary tree.
Output: the number of nodes that have a left child or a right child, but not both.
Initial call: `ONECHILD(root)`

```
 1: procedure ONECHILD(node)
 2:     if node = NULL then
 3:         return 0
 4:     end if
 5:     count <- ONECHILD(node.left)
 6:     count <- count + ONECHILD(node.right)
 7:     if node.left = NULL and node.right is not NULL then
 8:         count <- count + 1
 9:     end if
10:     if node.left is not NULL and node.right = NULL then
11:         count <- count + 1
12:     end if
13:     return count
14: end procedure
```

(b) Worked Example

With $n = 8$ there are $2n + 1 = 17$ calls.

#	Call	Lines executed	Returns
1	ONECHILD(5)	2, 5, 6, 7, 10, 13	1
2	ONECHILD(12)	2, 5, 6, 7, 10, 13	1
3	ONECHILD(21)	2, 5, 6, 7, 10, 11, 13	1
4	ONECHILD(4)	2, 5, 6, 7, 10, 13	0
5	ONECHILD(NULL), left of 4	2, 3	0
6	ONECHILD(NULL), right of 4	2, 3	0
7	ONECHILD(NULL), right of 21	2, 3	0
8	ONECHILD(3)	2, 5, 6, 7, 10, 13	0
9	ONECHILD(NULL), left of 3	2, 3	0
10	ONECHILD(NULL), right of 3	2, 3	0
11	ONECHILD(8)	2, 5, 6, 7, 10, 13	0
12	ONECHILD(17)	2, 5, 6, 7, 10, 13	0
13	ONECHILD(NULL), left of 17	2, 3	0
14	ONECHILD(NULL), right of 17	2, 3	0
15	ONECHILD(9)	2, 5, 6, 7, 10, 13	0
16	ONECHILD(NULL), left of 9	2, 3	0
17	ONECHILD(NULL), right of 9	2, 3	0

Final output: ONECHILD(5) returns 1. The only node that adds 1 is 21 (line 11, left child 4 and no right child), which matches the expected output.

(c) Frequency Count

Here k is the number of one-child nodes: $k = 1$ when n is even and $k = 0$ when n is odd (see the Twist). A tree built from an array never has a right child without a left child, so line 8 never runs.

Line	Operation	Cost	Frequency	Count for the example
2	Test node = NULL	1	$2n + 1$	17
3	Return 0 (NULL child)	1	$n + 1$	9
5	Recursive call on left child	1	n	8
6	Recursive call on right child	1	n	8
7	Test right child only	1	n	8
8	count $\leftarrow$ count + 1	1	0	0
10	Test left child only	1	n	8
11	count $\leftarrow$ count + 1	1	k	1
13	Return count	1	n	8
	Total $T(n)$		$8n + 2 + k$	67

$T(n) = (2n + 1) + (n + 1) + 5n + k = 8n + 2 + k$. Check: $T(8) = 8(8) + 2 + 1 = 67$, the same as the Count column.

(d) Final Complexity

The dominant term is $8n$. Best case $\Theta(n)$ and worst case $\Theta(n)$: every node and every NULL child is visited once, and k is at most 1. Extra space is the call stack, $\Theta(h)$, which is $\Theta(\log n)$ for a tree built from an array (it is complete) and $\Theta(n)$ in the worst case of a chain.

Twist

In a tree built from an array, the node at index i has a left child exactly when $2i + 1 \leq n - 1$ and a right child exactly when $2i + 2 \leq n - 1$. It has exactly one child when the first holds and the second does not, that is when $2i + 1 \leq n - 1 < 2i + 2$. Because these are whole numbers, this means $n - 1 = 2i + 1$, or $n = 2i + 2$. So there is a one-child node only if n is even, and then there is exactly one, at index $i = (n - 2)/2$; it is always a left-only child, because the left index $2i + 1$ is filled before the right index $2i + 2$. When n is odd, $n = 2i + 2$ has no solution, so no node has exactly one child. For $n = 8$: $i = 3$, which holds 21, as found above.

Question 4 (5 points): Are Two Trees Identical?

(a) Pseudocode

Two trees are identical when both are empty, or when both roots hold the same value and the left subtrees and the right subtrees are identical. The algorithm stops at the first difference.

Algorithm 4: Identical Trees
Input: pointers `a` and `b` to the roots of two binary trees.
Output: true if the trees have the same shape and the same value in every position, otherwise false.
Initial call: `IDENTICAL(rootA, rootB)`

```
 1: procedure IDENTICAL(a, b)
 2:     if a = NULL and b = NULL then
 3:         return true
 4:     end if
 5:     if a = NULL or b = NULL then
 6:         return false
 7:     end if
 8:     if a.data != b.data then
 9:         return false
10:     end if
11:     leftSame <- IDENTICAL(a.left, b.left)
12:     if leftSame = false then
13:         return false
14:     end if
15:     return IDENTICAL(a.right, b.right)
16: end procedure
```

(b) Worked Example

Trees A and B are both built from arr. Tree C is built from {5, 12, 8, 21, 3, 17, 10}, so it differs from A only in its last leaf:



IDENTICAL(A, B).

#	Call	Lines executed	Returns
1	IDENTICAL(5, 5)	2, 5, 8, 11, 12, 15	true
2	IDENTICAL(12, 12)	2, 5, 8, 11, 12, 15	true
3	IDENTICAL(21, 21)	2, 5, 8, 11, 12, 15	true
4	IDENTICAL(NULL, NULL), left of 21	2, 3	true
5	IDENTICAL(NULL, NULL), right of 21	2, 3	true
6	IDENTICAL(3, 3)	2, 5, 8, 11, 12, 15	true
7	IDENTICAL(NULL, NULL), left of 3	2, 3	true
8	IDENTICAL(NULL, NULL), right of 3	2, 3	true
9	IDENTICAL(8, 8)	2, 5, 8, 11, 12, 15	true
10	IDENTICAL(17, 17)	2, 5, 8, 11, 12, 15	true
11	IDENTICAL(NULL, NULL), left of 17	2, 3	true
12	IDENTICAL(NULL, NULL), right of 17	2, 3	true
13	IDENTICAL(9, 9)	2, 5, 8, 11, 12, 15	true
14	IDENTICAL(NULL, NULL), left of 9	2, 3	true
15	IDENTICAL(NULL, NULL), right of 9	2, 3	true

IDENTICAL(A, C).

#	Call	Lines executed	Returns
1	IDENTICAL(5, 5)	2, 5, 8, 11, 12, 15	false
2	IDENTICAL(12, 12)	2, 5, 8, 11, 12, 15	true
3	IDENTICAL(21, 21)	2, 5, 8, 11, 12, 15	true
4	IDENTICAL(NULL, NULL), left of 21	2, 3	true
5	IDENTICAL(NULL, NULL), right of 21	2, 3	true
6	IDENTICAL(3, 3)	2, 5, 8, 11, 12, 15	true
7	IDENTICAL(NULL, NULL), left of 3	2, 3	true
8	IDENTICAL(NULL, NULL), right of 3	2, 3	true
9	IDENTICAL(8, 8)	2, 5, 8, 11, 12, 15	false
10	IDENTICAL(17, 17)	2, 5, 8, 11, 12, 15	true
11	IDENTICAL(NULL, NULL), left of 17	2, 3	true
12	IDENTICAL(NULL, NULL), right of 17	2, 3	true
13	IDENTICAL(9, 10)	2, 5, 8, 9	false

Final output: IDENTICAL(A, B) = true and IDENTICAL(A, C) = false, which match the expected outputs.

(c) Frequency Count

The Frequency column is for two identical trees of n nodes each, which is the worst case (no early return).

Line	Operation	Cost	Frequency	Count (A, B)	Count (A, C)
2	Test a = NULL and b = NULL	1	$2n+1$	15	13
3	Return true (both NULL)	1	$n+1$	8	6
5	Test a = NULL or b = NULL	1	n	7	7
6	Return false (shape differs)	1	0	0	0
8	Test a.data $\neq$ b.data	1	n	7	7
9	Return false (value differs)	1	0	0	1
11	Recursive call on left children	1	n	7	6
12	Test leftSame = false	1	n	7	6
13	Return false (left differs)	1	0	0	0
15	Recursive call on right children	1	n	7	6
	Total $T(n)$		$8n+2$	58	52

$T(n) = (2n+1) + (n+1) + 5n = 8n+2$. Check: $T(7) = 8(7)+2 = 58$, the same as the Count (A, B) column. The run on A and C stops early and makes 13 calls instead of 15, for a total of 52.

(d) Final Complexity

The dominant term is $8n$. Worst case $\Theta(n)$: the trees are identical (or differ only at the last node in preorder), so every pair of nodes and every pair of NULL children is compared. Best case $\Theta(1)$: the two roots hold different values, so the first call returns false after 4 steps (lines 2, 5, 8, 9). Extra space is the call stack, $\Theta(h)$, which is $\Theta(n)$ for a chain.

Twist

IDENTICAL(A, C) first finds a difference at call 13, IDENTICAL(9, 10): the right child of 8, the last node of A in preorder (5, 12, 21, 3, 8, 17, 9). The algorithm compares in preorder, so it had to check every other pair first, which is why A and C cost almost as much as the worst case (52 against 58). The best case ($\Theta(1)$) and the worst case ($\Theta(n)$) for two trees of n nodes are explained in (d).

Question 5 (5 points): Minimum Value in a Binary Tree

(a) Pseudocode

The tree is not ordered, so the smallest value can be anywhere. The minimum of a tree is the smallest of the node's value and the minimums of its two subtrees. An empty subtree returns INFINITY so it never wins.

Algorithm 5: Minimum Value
Input: a pointer `node` to the root of a binary tree.
Output: the smallest value in the tree (INFINITY for an empty tree).
Initial call: `MINVALUE(root)`

```
1: procedure MINVALUE(node)
2:     if node = NULL then
3:         return INFINITY
4:     end if
5:     leftMin <- MINVALUE(node.left)
6:     rightMin <- MINVALUE(node.right)
7:     return MIN(node.data, leftMin, rightMin)
8: end procedure
```

(b) Worked Example

#	Call	Lines executed	Returns
1	MINVALUE(5)	2, 5, 6, 7	3
2	MINVALUE(12)	2, 5, 6, 7	3
3	MINVALUE(21)	2, 5, 6, 7	21
4	MINVALUE(NULL), left of 21	2, 3	∞
5	MINVALUE(NULL), right of 21	2, 3	∞
6	MINVALUE(3)	2, 5, 6, 7	3
7	MINVALUE(NULL), left of 3	2, 3	∞
8	MINVALUE(NULL), right of 3	2, 3	∞
9	MINVALUE(8)	2, 5, 6, 7	8
10	MINVALUE(17)	2, 5, 6, 7	17
11	MINVALUE(NULL), left of 17	2, 3	∞
12	MINVALUE(NULL), right of 17	2, 3	∞
13	MINVALUE(9)	2, 5, 6, 7	9
14	MINVALUE(NULL), left of 9	2, 3	∞
15	MINVALUE(NULL), right of 9	2, 3	∞

Final output: MINVALUE(5) returns $\min(5, 3, 8) = 3$, which matches the expected output.

(c) Frequency Count

Line	Operation	Cost	Frequency	Count for the example
2	Test node = NULL	1	$2n+1$	15
3	Return INFINITY (NULL child)	1	$n+1$	8
5	Recursive call on left child	1	n	7
6	Recursive call on right child	1	n	7
7	Return MIN of three values	1	n	7
	Total $T(n)$		$6n+2$	44

$T(n) = (2n+1) + (n+1) + 3n = 6n+2$. Check: $T(7) = 6(7)+2 = 44$, the same as the Count column.

(d) Final Complexity

The dominant term is $6n$. Best case $\Theta(n)$ and worst case $\Theta(n)$: without an ordering, the algorithm cannot skip any node, so every input of n nodes costs exactly $6n+2$. Extra space is the call stack, $\Theta(h)$, with the worst case $\Theta(n)$ for a chain.

Twist

In a binary search tree every smaller value is in the left subtree, so the minimum is the leftmost node: start at the root and follow left pointers until the left child is NULL. No recursion and no right subtrees are needed.

The loop takes one step per level on the way down, so the complexity drops from $\Theta(n)$ to $\Theta(h)$: best case $\Theta(1)$ (the root has no left child), $\Theta(\log n)$ for a balanced tree, and worst case $\Theta(n)$ for a chain of left children. The extra space drops to $\Theta(1)$, because the loop needs only the pointer `node`.

Question 6 (7 points): Invert (Mirror) a Binary Tree

(a) Pseudocode

Swap the two children of the current node, then invert both subtrees. Every node is handled once, so after the traversal every node has had its children swapped.

Algorithm 6: Invert a Binary Tree
Input: a pointer `node` to the root of a binary tree.
Output: the same root, with the tree changed into its mirror image.
Initial call: `INVERT(root)`

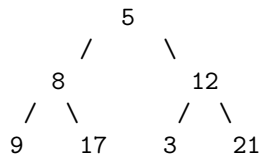
```
 1: procedure INVERT(node)
 2:     if node = NULL then
 3:         return NULL
 4:     end if
 5:     temp <- node.left
 6:     node.left <- node.right
 7:     node.right <- temp
 8:     INVERT(node.left)
 9:     INVERT(node.right)
10:     return node
11: end procedure
```

(b) Worked Example

The column "left / right after swap" shows the children of the node once lines 5 to 7 have run. Because the swap happens first, line 8 goes into the old right subtree.

#	Call	Lines executed	left / right after swap	Returns
1	INVERT(5)	2, 5, 6, 7, 8, 9, 10	8 / 12	node 5
2	INVERT(8)	2, 5, 6, 7, 8, 9, 10	9 / 17	node 8
3	INVERT(9)	2, 5, 6, 7, 8, 9, 10	NULL / NULL	node 9
4	INVERT(NULL), left of 9	2, 3	-	NULL
5	INVERT(NULL), right of 9	2, 3	-	NULL
6	INVERT(17)	2, 5, 6, 7, 8, 9, 10	NULL / NULL	node 17
7	INVERT(NULL), left of 17	2, 3	-	NULL
8	INVERT(NULL), right of 17	2, 3	-	NULL
9	INVERT(12)	2, 5, 6, 7, 8, 9, 10	3 / 21	node 12
10	INVERT(3)	2, 5, 6, 7, 8, 9, 10	NULL / NULL	node 3
11	INVERT(NULL), left of 3	2, 3	-	NULL
12	INVERT(NULL), right of 3	2, 3	-	NULL
13	INVERT(21)	2, 5, 6, 7, 8, 9, 10	NULL / NULL	node 21
14	INVERT(NULL), left of 21	2, 3	-	NULL
15	INVERT(NULL), right of 21	2, 3	-	NULL

Final tree:



Final output: level order 5, 8, 12, 9, 17, 3, 21, which matches the expected output.

(c) Frequency Count

Line	Operation	Cost	Frequency	Count for the example
2	Test node = NULL	1	$2n+1$	15
3	Return NULL	1	$n+1$	8
5	temp $\leftarrow$ node.left	1	n	7
6	node.left $\leftarrow$ node.right	1	n	7
7	node.right $\leftarrow$ temp	1	n	7
8	Recursive call on new left child	1	n	7
9	Recursive call on new right child	1	n	7
10	Return node	1	n	7
	Total $T(n)$		$9n+2$	65

$T(n) = (2n+1) + (n+1) + 6n = 9n+2$. Check: $T(7) = 9(7)+2 = 65$, the same as the Count column.

(d) Final Complexity

The dominant term is $9n$. Best case $\Theta(n)$ and worst case $\Theta(n)$: every node must be swapped, whatever the shape. Extra space is the call stack, $\Theta(h)$; the worst case is a chain, $\Theta(n)$.

Twist

Inorder before inverting: 21 12 3 5 17 8 9. Inorder after inverting: 9 8 17 5 3 12 21. The second list is the first one read backwards. Inorder visits Left, Node, Right. After the mirror, the new left subtree of every node is its old right subtree, so inorder on the mirror visits old Right, Node, old Left at every node. That is reverse inorder of the original tree, which visits exactly the same nodes in exactly the opposite order.

Question 7 (7 points): Print Each Level on Its Own Line, and the Right Side View

(a) Pseudocode

This is level order (rule T5) with one change: at the start of each pass of the while loop the queue holds exactly one whole level, so its size tells the for loop how many nodes to take before printing a newline. The last node taken in each level is the one seen from the right.

Algorithm 7: Levels and Right Side View
Input: a pointer `root` to the root of a binary tree.
Output: each level printed on its own line; the list of the last node of each level is returned.
Initial call: `LEVELS(root)`

```
 1: procedure LEVELS(root)
 2:     if root = NULL then
 3:         return empty list
 4:     end if
 5:     view <- empty list
 6:     Q <- empty queue
 7:     ENQUEUE(Q, root)
 8:     while Q is not empty do
 9:         size <- SIZEOF(Q)
10:         for i <- 1 to size do
11:             node <- DEQUEUE(Q)
12:             PRINT node.data
13:             if node.left is not NULL then
14:                 ENQUEUE(Q, node.left)
15:             end if
16:             if node.right is not NULL then
17:                 ENQUEUE(Q, node.right)
18:             end if
19:             if i = size then
20:                 APPEND(view, node.data)
21:             end if
22:         end for
23:         PRINT newline
24:     end while
25:     return view
26: end procedure
```

(b) Worked Example

One row per pass of the for loop. A new level starts when the while loop runs line 9 again.

Level	i	Dequeue	Enqueue	Queue after	Append	view
0	1 of 1	5	12, 8	12, 8	5	[5]
1	1 of 2	12	21, 3	8, 21, 3	-	[5]
1	2 of 2	8	17, 9	21, 3, 17, 9	8	[5, 8]
2	1 of 4	21	none	3, 17, 9	-	[5, 8]
2	2 of 4	3	none	17, 9	-	[5, 8]
2	3 of 4	17	none	9	-	[5, 8]
2	4 of 4	9	none	empty	9	[5, 8, 9]

Final output: the three printed lines are [5], [12, 8] and [21, 3, 17, 9], and the returned right side view is 5, 8, 9. Both match the expected output.

(c) Frequency Count

As in Question 2, lines 14 and 17 add to $n - 1$; the floor terms are for a tree built from an array.

Line	Operation	Cost	Frequency	Count for the example
2	Test root = NULL	1	1	1
3	Return empty list	1	0	0
5	view $\leftarrow$ empty list	1	1	1
6	Create empty queue	1	1	1
7	Enqueue root	1	1	1
8	Test Q is not empty	1	$h + 2$	4
9	size $\leftarrow$ SIZEOF(Q)	1	$h + 1$	3
10	Test of the for loop	1	$n + h + 1$	10
11	Dequeue node	1	n	7
12	Print node.data	1	n	7
13	Test node.left is not NULL	1	n	7
14	Enqueue left child	1	$\lfloor n/2 \rfloor$	3
16	Test node.right is not NULL	1	n	7
17	Enqueue right child	1	$\lfloor (n-1)/2 \rfloor$	3
19	Test i = size	1	n	7
20	Append last node of level to view	1	$h + 1$	3
23	Print newline	1	$h + 1$	3
25	Return view	1	1	1
	Total $T(n)$		$7n + 5h + 10$	69

$T(n) = 5 + (h + 2) + (h + 1) + (n + h + 1) + 5n + (n - 1) + 2(h + 1) = 7n + 5h + 10$. Check: $T(7) = 7(7) + 5(2) + 10 = 69$, the same as the Count column.

(d) Final Complexity

The dominant term is $7n$, because the number of levels is at most n. Best case $\Theta(n)$ (a complete tree, $h = \lfloor \log_2 n \rfloor$) and worst case $\Theta(n)$ (a chain, $h = n - 1$, which gives the largest count $12n + 5$). Extra space is the queue, $\Theta(w)$ for the widest level w: $\Theta(n)$ in the worst case of a complete tree, whose last level holds about $n/2$ nodes.

Twist

The terms in n come from work done once per node: dequeue, print, the two child tests, the test `i = size`, the $n - 1$ enqueues, and n of the tests of the for loop. The terms in $h + 1$ (the number of levels) come from work done once per level: `size <- SIZEOF(Q)`, the final failing test of the for loop, appending to the view, and printing the newline, each $h + 1$ times, plus the while test, which runs $h + 2$ times (once per level and once more to stop). That gives $4(h + 1) + (h + 2) = 5h + 6$ from the levels; together with the $7n$ and the constant steps, $T(n) = 7n + 5h + 10$.

Question 8 (7 points): Zigzag (Spiral) Level Order Traversal

(a) Pseudocode

Two stacks are used. The nodes of the current level are popped from `current` and their children are pushed onto `next`. A stack reverses order, so if a left-to-right level pushes left child then right child, the next level comes out right to left; a right-to-left level pushes right child then left child, so the level after it comes out left to right again.

Algorithm 8: Zigzag Level Order (two stacks)
Input: a pointer `root` to the root of a binary tree.
Output: the values printed level by level, alternating left to right and right to left, starting left to right.
Initial call: `ZIGZAG(root)`

```
 1: procedure ZIGZAG(root)
 2:     if root = NULL then
 3:         return
 4:     end if
 5:     current <- empty stack
 6:     next <- empty stack
 7:     leftToRight <- true
 8:     PUSH(current, root)
 9:     while current is not empty do
10:         while current is not empty do
11:             node <- POP(current)
12:             PRINT node.data
13:             if leftToRight = true then
14:                 if node.left is not NULL then
15:                     PUSH(next, node.left)
16:                 end if
17:                 if node.right is not NULL then
18:                     PUSH(next, node.right)
19:                 end if
20:             else
21:                 if node.right is not NULL then
22:                     PUSH(next, node.right)
23:                 end if
24:                 if node.left is not NULL then
25:                     PUSH(next, node.left)
26:                 end if
27:             end if
28:         end while
29:         SWAP(current, next)
30:         leftToRight <- not leftToRight
31:     end while
32: end procedure
```

(b) Worked Example

One row per pop, plus one row for each swap at the end of a level. Stacks are written bottom first, so the top is on the right.

Level	Direction	Pop	Push to next	current	next	Output
0	L to R	5	12, 8	empty	12, 8	5
0	swap	-	-	12, 8	empty	5
1	R to L	8	9, 17	12	9, 17	5 8
1	R to L	12	3, 21	empty	9, 17, 3, 21	5 8 12
1	swap	-	-	9, 17, 3, 21	empty	5 8 12
2	L to R	21	none	9, 17, 3	empty	5 8 12 21
2	L to R	3	none	9, 17	empty	5 8 12 21 3
2	L to R	17	none	9	empty	5 8 12 21 3 17
2	L to R	9	none	empty	empty	5 8 12 21 3 17 9
2	swap	-	-	empty	empty	5 8 12 21 3 17 9

Final output: 5, 8, 12, 21, 3, 17, 9, which matches the expected output.

(c) Frequency Count

The push lines depend on where the children are. Let n_e and n_o be the numbers of nodes on even and odd levels ($n_e + n_o = n$), and let ℓ_o, r_o and ℓ_e, r_e be the numbers of left and right children on odd and even levels (the root is not a child). Every node except the root is pushed exactly once, so $\ell_o + r_o + \ell_e + r_e = n - 1$. For the reference tree $n_e = 5$, $n_o = 2$, $\ell_o = r_o = 1$ (12 and 8) and $\ell_e = r_e = 2$ (21, 17 and 3, 9).

Line	Operation	Cost	Frequency	Count for the example
2	Test root = NULL	1	1	1
3	Return (empty tree)	1	0	0
5	current ← empty stack	1	1	1
6	next ← empty stack	1	1	1
7	leftToRight ← true	1	1	1
8	Push root	1	1	1
9	Outer test current not empty	1	$h + 2$	4
10	Inner test current not empty	1	$n + h + 1$	10
11	Pop node	1	n	7
12	Print node.data	1	n	7
13	Test leftToRight	1	n	7
14	Test left child (L to R level)	1	n_e	5
15	Push left child	1	ℓ_o	1
17	Test right child (L to R level)	1	n_e	5
18	Push right child	1	r_o	1
21	Test right child (R to L level)	1	n_o	2
22	Push right child	1	r_e	2
24	Test left child (R to L level)	1	n_o	2
25	Push left child	1	ℓ_e	2
29	Swap the two stacks	1	$h + 1$	3
30	Flip direction	1	$h + 1$	3
	Total $T(n)$		$7n + 4h + 9$	66

$T(n) = 5 + (h + 2) + (n + h + 1) + 3n + 2(n_e + n_o) + (n - 1) + 2(h + 1) = 7n + 4h + 9$. Check: $T(7) = 7(7) + 4(2) + 9 = 66$, the same as the Count column.

(d) Final Complexity

The dominant term is $7n$, since $h \leq n - 1$. Best case $\Theta(n)$ and worst case $\Theta(n)$: every node is pushed and popped exactly once whatever the shape (a chain gives the largest count, $11n + 5$). Extra space: the two stacks hold at most two levels at a time, $\Theta(w)$, which is $\Theta(n)$ in the worst case of a complete tree.

Twist

The second method is ordinary level order that collects each level in a list and reverses the list on odd levels before printing. REVERSE and PRINT row touch every element of the row, so they are counted once per element.

Algorithm 8b: Zigzag Level Order (queue plus reversing)
Input: a pointer `root` to the root of a binary tree.
Output: the same zigzag order as Algorithm 8.
Initial call: `ZIGZAGQ(root)`

```
 1: procedure ZIGZAGQ(root)
 2:     if root = NULL then
 3:         return
 4:     end if
 5:     Q <- empty queue
 6:     ENQUEUE(Q, root)
 7:     level <- 0
 8:     while Q is not empty do
 9:         size <- SIZEOF(Q)
10:         row <- empty list
11:         for i <- 1 to size do
12:             node <- DEQUEUE(Q)
13:             APPEND(row, node.data)
14:             if node.left is not NULL then
15:                 ENQUEUE(Q, node.left)
16:             end if
17:             if node.right is not NULL then
18:                 ENQUEUE(Q, node.right)
19:             end if
20:         end for
21:         if level mod 2 = 1 then
22:             REVERSE(row)
23:         end if
24:         PRINT row
25:         level <- level + 1
26:     end while
27: end procedure
```

Line	Operation	Cost	Frequency	Count for the example
2	Test root = NULL	1	1	1
3	Return (empty tree)	1	0	0
5	Create empty queue	1	1	1
6	Enqueue root	1	1	1
7	level $\leftarrow$ 0	1	1	1
8	Test Q is not empty	1	$h+2$	4
9	size $\leftarrow$ SIZEOF(Q)	1	$h+1$	3
10	row $\leftarrow$ empty list	1	$h+1$	3
11	Test of the for loop	1	$n+h+1$	10
12	Dequeue node	1	n	7
13	Append node.data to row	1	n	7
14	Test node.left is not NULL	1	n	7
15	Enqueue left child	1	$\lfloor n/2 \rfloor$	3
17	Test node.right is not NULL	1	n	7
18	Enqueue right child	1	$\lfloor (n-1)/2 \rfloor$	3
21	Test level mod 2 = 1	1	$h+1$	3
22	REVERSE row (per element)	1	n_o	2
24	PRINT row (per element)	1	n	7
25	level $\leftarrow$ level + 1	1	$h+1$	3
	Total $T(n)$		$7n + n_o + 6h + 10$	73

$T(n) = 7n + n_o + 6h + 10$, and $T(7) = 7(7) + 2 + 6(2) + 10 = 73$, the same as the Count column.

The two-stack method uses less extra work: 66 steps against 73 on the reference tree, and in general $(7n + n_o + 6h + 10) - (7n + 4h + 9) = n_o + 2h + 1 > 0$ for every tree. The stacks already produce each level in the order it is printed, while the queue method has to store every level in a list, test the level number, and move all n_o values of the odd levels a second time to reverse them. Both methods are $\Theta(n)$.

Question 9 (7 points): Is the Tree Complete?

(a) Pseudocode

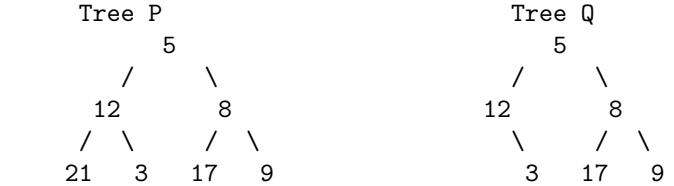
Level order is run with the NULL children put into the queue as well. In a complete tree all real nodes come before all NULLs in level order, so the tree is not complete as soon as a real node is dequeued after a NULL.

Algorithm 9: Complete Tree Test
Input: a pointer `root` to the root of a binary tree.
Output: true if the tree is complete, otherwise false.
Initial call: `COMPLETE(root)`

```
 1: procedure COMPLETE(root)
 2:     if root = NULL then
 3:         return true
 4:     end if
 5:     Q <- empty queue
 6:     ENQUEUE(Q, root)
 7:     seenNull <- false
 8:     while Q is not empty do
 9:         node <- DEQUEUE(Q)
10:         if node = NULL then
11:             seenNull <- true
12:         else
13:             if seenNull = true then
14:                 return false
15:             end if
16:             ENQUEUE(Q, node.left)
17:             ENQUEUE(Q, node.right)
18:         end if
19:     end while
20:     return true
21: end procedure
```

(b) Worked Example

In the queue, N stands for a NULL child. Tree Q has a gap where the left child of 12 should be:



COMPLETE(P).

Pass	Dequeue	seenNull	Enqueue	Queue after (front first)	Result
1	5	false	12, 8	12, 8	-
2	12	false	21, 3	8, 21, 3	-
3	8	false	17, 9	21, 3, 17, 9	-
4	21	false	N, N	3, 17, 9, N, N	-
5	3	false	N, N	17, 9, N, N, N, N	-
6	17	false	N, N	9, N, N, N, N, N, N	-
7	9	false	N, N	N, N, N, N, N, N, N, N	-
8	N	true	none	N, N, N, N, N, N, N	-
9	N	true	none	N, N, N, N, N, N	-
10	N	true	none	N, N, N, N, N	-
11	N	true	none	N, N, N, N	-
12	N	true	none	N, N, N	-
13	N	true	none	N, N	-
14	N	true	none	N	-
15	N	true	none	empty	-
16 (end)	-	true	-	empty	return true

COMPLETE(Q).

Pass	Dequeue	seenNull	Enqueue	Queue after (front first)	Result
1	5	false	12, 8	12, 8	-
2	12	false	N, 3	8, N, 3	-
3	8	false	17, 9	N, 3, 17, 9	-
4	N	true	none	3, 17, 9	-
5	3	true	none	17, 9	return false

Final output: COMPLETE(P) = true (all 7 nodes come out before the first N) and COMPLETE(Q) = false (node 3 comes out after the N that stands for the missing left child of 12). Both match the expected outputs.

(c) Frequency Count

The Frequency column is for a complete tree, the worst case: all n nodes and all $n+1$ NULLs pass through the queue.

Line	Operation	Cost	Frequency	Count for P	Count for Q
2	Test root = NULL	1	1	1	1
3	Return true (empty tree)	1	0	0	0
5	Create empty queue	1	1	1	1
6	Enqueue root	1	1	1	1
7	seenNull ← false	1	1	1	1
8	Test Q is not empty	1	$2n+2$	16	5
9	Dequeue item	1	$2n+1$	15	5
10	Test node = NULL	1	$2n+1$	15	5
11	seenNull ← true	1	$n+1$	8	1
13	Test seenNull = true	1	n	7	4
14	Return false	1	0	0	1
16	Enqueue left child (may be NULL)	1	n	7	3
17	Enqueue right child (may be NULL)	1	n	7	3
20	Return true	1	1	1	0
	Total $T(n)$		$10n+10$	80	31

$T(n) = 4 + (2n+2) + 2(2n+1) + (n+1) + 3n + 1 = 10n + 10$. Check: $T(7) = 10(7) + 10 = 80$, the same as the Count for P column. Tree Q ($n = 6$) stops on pass 5, after 31 steps.

(d) Final Complexity

The dominant term is $10n$. Worst case $\Theta(n)$: a complete tree, where every node and every NULL child is enqueued and dequeued. Best case $\Theta(1)$: the root has no left child but has a right child, so the answer false is found on the third pass. Extra space is the queue, $\Theta(w)$ including NULLs, which is $\Theta(n)$ in the worst case (the last level and its NULL children).

Twist

A tree is complete exactly when its nodes fill level-order positions $0, 1, \ldots, n-1$ with no gap. INSERT puts the new node at the first free position in level order, which is position n (rule I6), so after it the positions $0, \ldots, n$ are filled with no gap. DELETE never makes a hole in the middle: it copies the deepest, right-most node (position $n-1$, the last one in level order) into the target and removes that last position (rules D5 and D6), so the positions $0, \ldots, n-2$ stay filled. Each operation only ever adds or removes the last position, so the tree stays complete (rule C6).

Question 10 (7 points): Lowest Common Ancestor

(a) Pseudocode

A single recursive pass. A call returns the node if it holds a or b, otherwise it asks both subtrees. If both subtrees return a non-NULL answer, a is on one side and b on the other, so the current node is the lowest common ancestor. If only one side is non-NULL, that answer is passed up. Both a and b are assumed to be in the tree.

Algorithm 10: Lowest Common Ancestor (single pass)
Input: a pointer `node` to the root of a binary tree and two values a and b that are in the tree.
Output: the deepest node that has both a and b in its subtree.
Initial call: `LCA(root, a, b)`

```
 1: procedure LCA(node, a, b)
 2:     if node = NULL then
 3:         return NULL
 4:     end if
 5:     if node.data = a or node.data = b then
 6:         return node
 7:     end if
 8:     leftAns <- LCA(node.left, a, b)
 9:     rightAns <- LCA(node.right, a, b)
10:     if leftAns is not NULL and rightAns is not NULL then
11:         return node
12:     end if
13:     if leftAns is not NULL then
14:         return leftAns
15:     end if
16:     return rightAns
17: end procedure
```

(b) Worked Example

LCA(21, 3).

#	Call	Lines executed	Returns
1	LCA(5)	2, 5, 8, 9, 10, 13, 14	node 12
2	LCA(12)	2, 5, 8, 9, 10, 11	node 12
3	LCA(21)	2, 5, 6	node 21
4	LCA(3)	2, 5, 6	node 3
5	LCA(8)	2, 5, 8, 9, 10, 13, 16	NULL
6	LCA(17)	2, 5, 8, 9, 10, 13, 16	NULL
7	LCA(NULL), left of 17	2, 3	NULL
8	LCA(NULL), right of 17	2, 3	NULL
9	LCA(9)	2, 5, 8, 9, 10, 13, 16	NULL
10	LCA(NULL), left of 9	2, 3	NULL
11	LCA(NULL), right of 9	2, 3	NULL

LCA(21, 9).

#	Call	Lines executed	Returns
1	LCA(5)	2, 5, 8, 9, 10, 11	node 5
2	LCA(12)	2, 5, 8, 9, 10, 13, 14	node 21
3	LCA(21)	2, 5, 6	node 21
4	LCA(3)	2, 5, 8, 9, 10, 13, 16	NULL
5	LCA(NULL), left of 3	2, 3	NULL
6	LCA(NULL), right of 3	2, 3	NULL
7	LCA(8)	2, 5, 8, 9, 10, 13, 16	node 9
8	LCA(17)	2, 5, 8, 9, 10, 13, 16	NULL
9	LCA(NULL), left of 17	2, 3	NULL
10	LCA(NULL), right of 17	2, 3	NULL
11	LCA(9)	2, 5, 6	node 9

LCA(17, 8). The search stops at 8 (line 6), so 17 and 9 are never visited. Returning 8 is correct, because 8 is in its own subtree and 17 is below it.

#	Call	Lines executed	Returns
1	LCA(5)	2, 5, 8, 9, 10, 13, 16	node 8
2	LCA(12)	2, 5, 8, 9, 10, 13, 16	NULL
3	LCA(21)	2, 5, 8, 9, 10, 13, 16	NULL
4	LCA(NULL), left of 21	2, 3	NULL
5	LCA(NULL), right of 21	2, 3	NULL
6	LCA(3)	2, 5, 8, 9, 10, 13, 16	NULL
7	LCA(NULL), left of 3	2, 3	NULL
8	LCA(NULL), right of 3	2, 3	NULL
9	LCA(8)	2, 5, 6	node 8

Final output: LCA(21, 3) = 12, LCA(21, 9) = 5 and LCA(17, 8) = 8, which match the expected outputs.

(c) Frequency Count

The Frequency column is for the worst case, where a and b are both leaves: every node is visited, and only the 4 NULL children of a and b are skipped. m is the number of nodes that pass up an answer from the left only (line 14); lines 14 and 16 always add to $n - 3$. For LCA(21, 3) and LCA(21, 9), $m = 1$ (node 5 and node 12).

Line	Operation	Cost	Frequency	LCA(21, 3)	LCA(21, 9)	LCA(17, 8)
2	Test node = NULL	1	$2n - 3$	11	11	9
3	Return NULL	1	$n - 3$	4	4	4
5	Test node.data = a or b	1	n	7	7	5
6	Return node (found a or b)	1	2	2	2	1
8	Recursive call on left child	1	$n - 2$	5	5	4
9	Recursive call on right child	1	$n - 2$	5	5	4
10	Test both answers non-NULL	1	$n - 2$	5	5	4
11	Return node (the LCA)	1	1	1	1	0
13	Test leftAns is not NULL	1	$n - 3$	4	4	4
14	Return leftAns	1	m	1	1	0
16	Return rightAns	1	$n - 3 - m$	3	3	4
	Total $T(n)$		$9n - 15$	48	48	39

$T(n) = (2n-3)+(n-3)+n+2+3(n-2)+1+(n-3)+(n-3) = 9n-15$. Check: $T(7) = 9(7)-15 = 48$, the same as both worst-case columns. LCA(17, 8) costs only 39 because 8 is an internal node, so its subtree is never searched.

(d) Final Complexity

The dominant term is $9n$. Worst case $\Theta(n)$: a and b are both leaves, so the search reaches every node. Best case $\Theta(1)$: a or b is the root, so the first call returns at line 6 after 3 steps (lines 2, 5, 6). Extra space is the call stack, $\Theta(h)$, which is $\Theta(n)$ for a chain.

Twist

Method (1) finds the root-to-node path of each value with FINDPATH and returns the last node the two paths have in common.

Algorithm 10b: Lowest Common Ancestor with FINDPATH
Input: a pointer `root` to the root of a binary tree and two values a and b that are in the tree.
Output: the lowest common ancestor of a and b.
Initial call: `LCAPATH(root, a, b)`

```
 1: procedure FINDPATH(node, key, path)
 2:     if node = NULL then
 3:         return false
 4:     end if
 5:     APPEND(path, node)
 6:     if node.data = key then
 7:         return true
 8:     end if
 9:     if FINDPATH(node.left, key, path) = true then
10:         return true
11:     end if
12:     if FINDPATH(node.right, key, path) = true then
13:         return true
14:     end if
15:     REMOVELAST(path)
16:     return false
17: end procedure
18: procedure LCAPATH(root, a, b)
19:     pathA <- empty list
20:     pathB <- empty list
21:     FINDPATH(root, a, pathA)
22:     FINDPATH(root, b, pathB)
23:     i <- 0
24:     while i < SIZEOF(pathA) and i < SIZEOF(pathB) and pathA[i] = pathB[i] do
25:         i <- i + 1
26:     end while
27:     return pathA[i - 1]
28: end procedure
```

The paths found are [5, 12, 21] and [5, 12, 3] for LCA(21, 3), [5, 12, 21] and [5, 8, 9] for LCA(21, 9), and [5, 8, 17] and [5, 8] for LCA(17, 8). The last common entries are 12, 5 and 8. The frequency counts of the two methods on the same three inputs:

Line	Operation	Cost	LCA(21, 3)	LCA(21, 9)	LCA(17, 8)
2	Test node = NULL	1	9	16	19
3	Return false (NULL)	1	2	6	8
5	Append node to path	1	7	10	11
6	Test node.data = key	1	7	10	11
7	Return true (key found)	1	2	2	2
9	Search left subtree	1	5	8	9
10	Return true (found on left)	1	3	2	1
12	Search right subtree	1	2	6	8
13	Return true (found on right)	1	1	2	2
15	Remove node from path	1	1	4	6
16	Return false (not below node)	1	1	4	6
19	pathA $\leftarrow$ empty list	1	1	1	1
20	pathB $\leftarrow$ empty list	1	1	1	1
21	FINDPATH for a	1	1	1	1
22	FINDPATH for b	1	1	1	1
23	i $\leftarrow$ 0	1	1	1	1
24	Test common prefix	1	3	2	3
25	i $\leftarrow$ i + 1	1	2	1	2
27	Return pathA[i - 1]	1	1	1	1
	Total, path method		51	79	94
	Total, single pass (table above)		48	48	39

The single pass is cheaper on all three inputs. FINDPATH can visit every node (FINDPATH(9) searches the whole tree, because 9 is the last node in preorder), and method (1) runs it twice and then compares the paths, so in the worst case it does about two full traversals ($\approx 18n$ steps) against one for the single pass ($9n - 15$). Both methods are $\Theta(n)$, but method (1) also needs two extra lists of up to $h + 1$ nodes each, while method (2) needs only the call stack.

Question 11 (8 points): Build a Tree From Its Preorder and Inorder Traversals

(a) Pseudocode

The next unused preorder value is always the root of the part being built. Its position k in the inorder list splits that part into the left subtree (in[low..k-1]) and the right subtree (in[k+1..high]). The left subtree is built first, because preorder lists the whole left subtree before the right one.

Algorithm 11: Build Tree From Preorder and Inorder
Input: arrays pre[0..n-1] and in[0..n-1] holding the preorder and inorder traversals of a tree with distinct values, and the range low..high of in that the call must build.
Output: the root of the rebuilt tree.
Initial call: `preIndex <- 0`, then `BUILD(pre, in, 0, n - 1)` (preIndex is shared by all calls).

```
 1: procedure BUILD(pre, in, low, high)
 2:     if low > high then
 3:         return NULL
 4:     end if
 5:     node <- NEWNODE(pre[preIndex])
 6:     preIndex <- preIndex + 1
 7:     k <- low
 8:     while in[k] != node.data do
 9:         k <- k + 1
10:     end while
11:     node.left <- BUILD(pre, in, low, k - 1)
12:     node.right <- BUILD(pre, in, k + 1, high)
13:     return node
14: end procedure
```

(b) Worked Example

pre = 5, 12, 21, 3, 8, 17, 9 and in = 21, 12, 3, 5, 17, 8, 9 (indices 0 to 6). The search steps are the passes of line 9.

#	Call	Lines executed	Root	k (search steps)	Returns
1	BUILD(0, 6)	2, 5, 6, 7, 8 (x4), 9 (x3), 11, 12, 13	5	3 (3 steps)	node 5
2	BUILD(0, 2)	2, 5, 6, 7, 8 (x2), 9, 11, 12, 13	12	1 (1 step)	node 12
3	BUILD(0, 0)	2, 5, 6, 7, 8, 11, 12, 13	21	0 (0 steps)	node 21
4	BUILD(0, -1)	2, 3	-	-	NULL
5	BUILD(1, 0)	2, 3	-	-	NULL
6	BUILD(2, 2)	2, 5, 6, 7, 8, 11, 12, 13	3	2 (0 steps)	node 3
7	BUILD(2, 1)	2, 3	-	-	NULL
8	BUILD(3, 2)	2, 3	-	-	NULL
9	BUILD(4, 6)	2, 5, 6, 7, 8 (x2), 9, 11, 12, 13	8	5 (1 step)	node 8
10	BUILD(4, 4)	2, 5, 6, 7, 8, 11, 12, 13	17	4 (0 steps)	node 17
11	BUILD(4, 3)	2, 3	-	-	NULL
12	BUILD(5, 4)	2, 3	-	-	NULL
13	BUILD(6, 6)	2, 5, 6, 7, 8, 11, 12, 13	9	6 (0 steps)	node 9
14	BUILD(6, 5)	2, 3	-	-	NULL
15	BUILD(7, 6)	2, 3	-	-	NULL

Final output: root 5 with left subtree 12 (children 21, 3) and right subtree 8 (children 17, 9). Its level order is 5, 12, 8, 21, 3, 17, 9 and its postorder is 21, 3, 12, 17, 9, 8, 5, which match the expected check, so the tree from arr was rebuilt.

(c) Frequency Count

S is the total number of search steps, the sum of $k - low$ over all calls. For the example $S = 3 + 1 + 1 = 5$.

Line	Operation	Cost	Frequency	Count for the example
2	Test low > high	1	$2n + 1$	15
3	Return NULL (empty range)	1	$n + 1$	8
5	Create node from pre[preIndex]	1	n	7
6	preIndex ← preIndex + 1	1	n	7
7	k ← low	1	n	7
8	Test in[k] ≠ node.data	1	$n + S$	12
9	k ← k + 1	1	S	5
11	Recursive call for left part	1	n	7
12	Recursive call for right part	1	n	7
13	Return node	1	n	7
	Total $T(n)$		$10n + 2 + 2S$	82

$T(n) = (2n+1)+(n+1)+3n+(n+S)+S+3n = 10n+2+2S$. Check: $T(7) = 10(7)+2+2(5) = 82$, the same as the Count column.

(d) Final Complexity

S decides the complexity. Worst case: every root is the last value of its inorder range, which happens for a chain of left children (inorder is preorder reversed). The searches then take $(n-1)+(n-2)+\cdots+0 = n(n-1)/2$ steps, so $T(n) = n^2 + 9n + 2 = \Theta(n^2)$. Best case: every root is the first value of its range (a chain of right children), so $S = 0$ and $T(n) = 10n + 2 = \Theta(n)$. Extra space is the call stack, $\Theta(h)$, which is $\Theta(n)$ in the worst case.

Twist

Searching the inorder list each time gives the worst-case count $n^2 + 9n + 2$ shown above. A lookup table pos, filled once so that pos[v] is the inorder index of value v, replaces the whole search loop (lines 7 to 10) with one step.

Algorithm 11b: Build Tree With a Lookup Table
Input: arrays pre[0..n-1] and in[0..n-1] as before.
Output: the root of the rebuilt tree.
Initial call: `BUILDFAST(pre, in, n)` (pos and preIndex are shared by all calls).

```
 1: procedure BUILDFAST(pre, in, n)
 2:     for i <- 0 to n - 1 do
 3:         pos[in[i]] <- i
 4:     end for
 5:     preIndex <- 0
 6:     return BUILD2(pre, 0, n - 1)
 7: end procedure
 8: procedure BUILD2(pre, low, high)
 9:     if low > high then
10:         return NULL
11:     end if
12:     node <- NEWNODE(pre[preIndex])
13:     preIndex <- preIndex + 1
14:     k <- pos[node.data]
15:     node.left <- BUILD2(pre, low, k - 1)
16:     node.right <- BUILD2(pre, k + 1, high)
17:     return node
18: end procedure
```

Line	Operation	Cost	Frequency	Count for the example
2	Test of the for loop	1	$n+1$	8
3	pos[in[i]] $\leftarrow$ i	1	n	7
5	preIndex $\leftarrow$ 0	1	1	1
6	First call of BUILD2	1	1	1
9	Test low $>$ high	1	$2n+1$	15
10	Return NULL (empty range)	1	$n+1$	8
12	Create node from pre[preIndex]	1	n	7
13	preIndex $\leftarrow$ preIndex + 1	1	n	7
14	k $\leftarrow$ pos[node.data]	1	n	7
15	Recursive call for left part	1	n	7
16	Recursive call for right part	1	n	7
17	Return node	1	n	7
	Total $T(n)$		$11n+5$	82

$T(n) = (n+1) + n + 2 + (2n+1) + (n+1) + 6n = 11n + 5$, and $T(7) = 11(7) + 5 = 82$, the same as the Count column. This count no longer depends on the shape of the tree, so it is $\Theta(n)$ in the best and the worst case. On a chain of left children the search version grows as n^2, while the table version stays at $11n + 5$. The price is $\Theta(n)$ extra space for the table.

Question 12 (8 points): Diameter of a Binary Tree

(a) Pseudocode

The longest path that has a given node as its highest point goes down $leftH + 1$ edges on the left and $rightH + 1$ edges on the right, where the height of an empty subtree is -1. One postorder pass returns the height of each subtree and, at the same node, compares $leftH + rightH + 2$ with the best path found so far.

Algorithm 12: Diameter (height and diameter in one pass)
Input: a pointer `node` to the root of a binary tree.
Output: HEIGHTDIAM returns the height of the subtree; when the initial call finishes, `diameter` holds the number of edges on the longest path between any two nodes.
Initial call: `diameter <- 0`, then `HEIGHTDIAM(root)`; the answer is `diameter` (shared by all calls).

```
1: procedure HEIGHTDIAM(node)
2:     if node = NULL then
3:         return -1
4:     end if
5:     leftH <- HEIGHTDIAM(node.left)
6:     rightH <- HEIGHTDIAM(node.right)
7:     diameter <- MAX(diameter, leftH + rightH + 2)
8:     return 1 + MAX(leftH, rightH)
9: end procedure
```

(b) Worked Example

Path is $leftH + rightH + 2$, the longest path whose highest node is the current node; Diameter is the value after line 7 of that call; Returns is the height of the subtree.

#	Call	Lines	leftH, rightH	Path	Diameter	Returns
1	HEIGHTDIAM(5)	2, 5, 6, 7, 8	1, 1	4	4	2
2	HEIGHTDIAM(12)	2, 5, 6, 7, 8	0, 0	2	2	1
3	HEIGHTDIAM(21)	2, 5, 6, 7, 8	-1, -1	0	0	0
4	HEIGHTDIAM(NULL), left of 21	2, 3	-	-	-	-1
5	HEIGHTDIAM(NULL), right of 21	2, 3	-	-	-	-1
6	HEIGHTDIAM(3)	2, 5, 6, 7, 8	-1, -1	0	0	0
7	HEIGHTDIAM(NULL), left of 3	2, 3	-	-	-	-1
8	HEIGHTDIAM(NULL), right of 3	2, 3	-	-	-	-1
9	HEIGHTDIAM(8)	2, 5, 6, 7, 8	0, 0	2	2	1
10	HEIGHTDIAM(17)	2, 5, 6, 7, 8	-1, -1	0	2	0
11	HEIGHTDIAM(NULL), left of 17	2, 3	-	-	-	-1
12	HEIGHTDIAM(NULL), right of 17	2, 3	-	-	-	-1
13	HEIGHTDIAM(9)	2, 5, 6, 7, 8	-1, -1	0	2	0
14	HEIGHTDIAM(NULL), left of 9	2, 3	-	-	-	-1
15	HEIGHTDIAM(NULL), right of 9	2, 3	-	-	-	-1

Final output: diameter $= 4$, set at the root from $1 + 1 + 2$, which matches the expected output (one such path is 21, 12, 5, 8, 17).

(c) Frequency Count

Line	Operation	Cost	Frequency	Count for the example
2	Test node = NULL	1	$2n+1$	15
3	Return -1 (NULL child)	1	$n+1$	8
5	Recursive call on left child	1	n	7
6	Recursive call on right child	1	n	7
7	Update diameter with MAX	1	n	7
8	Return height 1 + MAX	1	n	7
	Total $T(n)$		$7n+2$	51

$T(n) = (2n+1) + (n+1) + 4n = 7n+2$. Check: $T(7) = 7(7) + 2 = 51$, the same as the Count column.

(d) Final Complexity

The dominant term is $7n$. Best case $\Theta(n)$ and worst case $\Theta(n)$: the count $7n+2$ does not depend on the shape at all. Extra space is the call stack, $\Theta(h)$, with the worst case $\Theta(n)$ for a chain.

Twist

HEIGHTDIAM computes the height and the diameter in the same pass: the height it needs at each node is the value the recursive calls return anyway, so no separate HEIGHT traversal is made. Every line in the table above has frequency $2n+1$, $n+1$ or n, none of which depends on the shape, so $T(n) = 7n+2$ for every tree, the chain included (a chain of 7 nodes also costs exactly 51 steps). The dominant term is $7n$, so the one-pass solution is $\Theta(n)$.

Question 13 (8 points): Serialize and Deserialize a Binary Tree

(a) Pseudocode

SERIALIZE writes the tree in preorder and writes the marker N for every NULL child. DESERIALIZE reads the list in the same preorder: each value it reads becomes a node whose left and right subtrees are read next, and each N it reads is an empty subtree.

Algorithm 13a: Serialize
Input: a pointer `node` to the root of a binary tree and the output list `out`.
Output: `out` holds the preorder values of the tree with N for every NULL child.
Initial call: `out <- empty list`, then `SERIALIZE(root, out)`

```
 1: procedure SERIALIZE(node, out)
 2:     if node = NULL then
 3:         APPEND(out, N)
 4:         return
 5:     end if
 6:     APPEND(out, node.data)
 7:     SERIALIZE(node.left, out)
 8:     SERIALIZE(node.right, out)
 9: end procedure
```

Algorithm 13b: Deserialize
Input: a list produced by SERIALIZE and a read position `idx` shared by all calls.
Output: the root of a tree identical to the one that was serialized.
Initial call: `idx <- 0`, then `root <- DESERIALIZE(list)`

```
 1: procedure DESERIALIZE(list)
 2:     value <- list[idx]
 3:     idx <- idx + 1
 4:     if value = N then
 5:         return NULL
 6:     end if
 7:     node <- NEWNODE(value)
 8:     node.left <- DESERIALIZE(list)
 9:     node.right <- DESERIALIZE(list)
10:     return node
11: end procedure
```

(b) Worked Example

SERIALIZE.

#	Call	Lines	Appends	out so far
1	SERIALIZE(5)	2, 6, 7, 8	5	5
2	SERIALIZE(12)	2, 6, 7, 8	12	5 12
3	SERIALIZE(21)	2, 6, 7, 8	21	5 12 21
4	SERIALIZE(NULL), left of 21	2, 3, 4	N	5 12 21 N
5	SERIALIZE(NULL), right of 21	2, 3, 4	N	5 12 21 N N
6	SERIALIZE(3)	2, 6, 7, 8	3	5 12 21 N N 3
7	SERIALIZE(NULL), left of 3	2, 3, 4	N	5 12 21 N N 3 N
8	SERIALIZE(NULL), right of 3	2, 3, 4	N	5 12 21 N N 3 N N
9	SERIALIZE(8)	2, 6, 7, 8	8	5 12 21 N N 3 N N 8
10	SERIALIZE(17)	2, 6, 7, 8	17	5 12 21 N N 3 N N 8 17
11	SERIALIZE(NULL), left of 17	2, 3, 4	N	5 12 21 N N 3 N N 8 17 N
12	SERIALIZE(NULL), right of 17	2, 3, 4	N	5 12 21 N N 3 N N 8 17 N N
13	SERIALIZE(9)	2, 6, 7, 8	9	5 12 21 N N 3 N N 8 17 N N 9
14	SERIALIZE(NULL), left of 9	2, 3, 4	N	5 12 21 N N 3 N N 8 17 N N 9 N
15	SERIALIZE(NULL), right of 9	2, 3, 4	N	5 12 21 N N 3 N N 8 17 N N 9 N N

DESERIALIZE on the list 5 12 21 N N 3 N N 8 17 N N 9 N N.

#	Call	Lines executed	idx	value	Returns
1	DESERIALIZE(list)	2, 3, 4, 7, 8, 9, 10	0	5	node 5
2	DESERIALIZE(list)	2, 3, 4, 7, 8, 9, 10	1	12	node 12
3	DESERIALIZE(list)	2, 3, 4, 7, 8, 9, 10	2	21	node 21
4	DESERIALIZE(list)	2, 3, 4, 5	3	N	NULL
5	DESERIALIZE(list)	2, 3, 4, 5	4	N	NULL
6	DESERIALIZE(list)	2, 3, 4, 7, 8, 9, 10	5	3	node 3
7	DESERIALIZE(list)	2, 3, 4, 5	6	N	NULL
8	DESERIALIZE(list)	2, 3, 4, 5	7	N	NULL
9	DESERIALIZE(list)	2, 3, 4, 7, 8, 9, 10	8	8	node 8
10	DESERIALIZE(list)	2, 3, 4, 7, 8, 9, 10	9	17	node 17
11	DESERIALIZE(list)	2, 3, 4, 5	10	N	NULL
12	DESERIALIZE(list)	2, 3, 4, 5	11	N	NULL
13	DESERIALIZE(list)	2, 3, 4, 7, 8, 9, 10	12	9	node 9
14	DESERIALIZE(list)	2, 3, 4, 5	13	N	NULL
15	DESERIALIZE(list)	2, 3, 4, 5	14	N	NULL

Final output: SERIALIZE produces 5 12 21 N N 3 N N 8 17 N N 9 N N, which matches the expected output. DESERIALIZE reads all 15 items, returns node 5 with children 12 (21, 3) and 8 (17, 9), and this tree has level order 5, 12, 8, 21, 3, 17, 9, so it is the tree from arr.

(c) Frequency Count

SERIALIZE.

Line	Operation	Cost	Frequency	Count for the example
2	Test node = NULL	1	$2n+1$	15
3	Append marker N	1	$n+1$	8
4	Return (NULL child)	1	$n+1$	8
6	Append node.data	1	n	7
7	Recursive call on left child	1	n	7
8	Recursive call on right child	1	n	7
	Total $T(n)$		$7n+3$	52

$T(n) = (2n + 1) + 2(n + 1) + 3n = 7n + 3$. Check: $T(7) = 7(7) + 3 = 52$, the same as the Count column.

DESERIALIZE.

Line	Operation	Cost	Frequency	Count for the example
2	value ← list[idx]	1	$2n + 1$	15
3	idx ← idx + 1	1	$2n + 1$	15
4	Test value = N	1	$2n + 1$	15
5	Return NULL	1	$n + 1$	8
7	Create node	1	n	7
8	Recursive call for left child	1	n	7
9	Recursive call for right child	1	n	7
10	Return node	1	n	7
	Total $T(n)$		$11n + 4$	81

$T(n) = 3(2n + 1) + (n + 1) + 4n = 11n + 4$. Check: $T(7) = 11(7) + 4 = 81$, the same as the Count column.

(d) Final Complexity

The dominant terms are $7n$ and $11n$. For both procedures the best case and the worst case are $\Theta(n)$, because they always handle all $2n + 1$ items. Extra space is the call stack, $\Theta(h)$, with the worst case $\Theta(n)$ for a chain; the list itself ($2n + 1$ items) is the output.

Twist

The list has one value for each of the n nodes and one N for each NULL child pointer, and a tree with n nodes has $n + 1$ NULL child pointers, so it holds $n + (n + 1) = 2n + 1$ items (15 for the reference tree). Each call of SERIALIZE appends exactly one item and each call of DESERIALIZE reads exactly one item, so each procedure is called exactly $2n + 1$ times. That gives the frequencies in both tables: $2n + 1$ for the lines run by every call (line 2 of SERIALIZE; lines 2, 3 and 4 of DESERIALIZE), $n + 1$ for the lines run only for an N, and n for the lines run only for a real value.

Question 14 (8 points): Is the Tree a Valid Binary Search Tree?

(a) Pseudocode

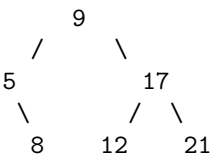
Every node must lie strictly inside an allowed range (low, high). The root may hold any value. Going left, the node's value becomes the new upper bound; going right, it becomes the new lower bound. This checks each node against all of its ancestors, not only against its parent.

Algorithm 14: Binary Search Tree Test (allowed range)
Input: a pointer `node` to the root of a binary tree and the allowed range `low`, `high`.
Output: true if every node is larger than all values in its left subtree and smaller than all values in its right subtree.
Initial call: `ISBST(root, -INFINITY, +INFINITY)`

```
 1: procedure ISBST(node, low, high)
 2:     if node = NULL then
 3:         return true
 4:     end if
 5:     if node.data <= low or node.data >= high then
 6:         return false
 7:     end if
 8:     leftOK <- ISBST(node.left, low, node.data)
 9:     if leftOK = false then
10:         return false
11:     end if
12:     return ISBST(node.right, node.data, high)
13: end procedure
```

(b) Worked Example

Tree S, given in level order as {9, 5, 17, 3, 8, 12, 21}:



ISBST(S). Each call shows (node, low, high).

#	Call	Lines executed	Returns
1	ISBST(9, $-\infty$, ∞)	2, 5, 8, 9, 12	true
2	ISBST(5, $-\infty$, 9)	2, 5, 8, 9, 12	true
3	ISBST(3, $-\infty$, 5)	2, 5, 8, 9, 12	true
4	ISBST(NULL, $-\infty$, 3), left of 3	2, 3	true
5	ISBST(NULL, 3, 5), right of 3	2, 3	true
6	ISBST(8, 5, 9)	2, 5, 8, 9, 12	true
7	ISBST(NULL, 5, 8), left of 8	2, 3	true
8	ISBST(NULL, 8, 9), right of 8	2, 3	true
9	ISBST(17, 9, ∞)	2, 5, 8, 9, 12	true
10	ISBST(12, 9, 17)	2, 5, 8, 9, 12	true
11	ISBST(NULL, 9, 12), left of 12	2, 3	true
12	ISBST(NULL, 12, 17), right of 12	2, 3	true
13	ISBST(21, 17, ∞)	2, 5, 8, 9, 12	true
14	ISBST(NULL, 17, 21), left of 21	2, 3	true
15	ISBST(NULL, 21, ∞), right of 21	2, 3	true

ISBST(P), where P is the tree from arr.

#	Call	Lines executed	Returns
1	ISBST(5, $-\infty$, ∞)	2, 5, 8, 9, 10	false
2	ISBST(12, $-\infty$, 5)	2, 5, 6	false

Final output: ISBST(S) = true and ISBST(P) = false (12 is in the left subtree of 5 but is not smaller than 5), which match the expected outputs.

(c) Frequency Count

The Frequency column is for a valid binary search tree, the worst case.

Line	Operation	Cost	Frequency	Count for S	Count for P
2	Test node = NULL	1	$2n+1$	15	2
3	Return true (NULL child)	1	$n+1$	8	0
5	Test node.data outside (low, high)	1	n	7	2
6	Return false (out of range)	1	0	0	1
8	Recursive call on left child	1	n	7	1
9	Test leftOK = false	1	n	7	1
10	Return false (left failed)	1	0	0	1
12	Recursive call on right child	1	n	7	0
	Total $T(n)$		$7n+2$	51	8

$T(n) = (2n+1) + (n+1) + 4n = 7n+2$. Check: $T(7) = 7(7)+2 = 51$, the same as the Count for S column. Tree P stops after 2 calls and 8 steps.

(d) Final Complexity

The dominant term is $7n$. Worst case $\Theta(n)$: the tree is a valid binary search tree, so every node and every NULL child is checked. Best case $\Theta(1)$: a child of the root is already out of range, as in P. Extra space is the call stack, $\Theta(h)$, with the worst case $\Theta(n)$ for a chain.

Twist

The inorder traversal of a binary search tree is strictly increasing, so the second solution walks the tree in inorder and compares each node with the previous value visited.

Algorithm 14b: Binary Search Tree Test (inorder)
Input: a pointer `node` to the root of a binary tree and the last value seen, `prev`, shared by all calls.
Output: true if the inorder values are strictly increasing, otherwise false.
Initial call: `prev <- -INFINITY`, then `ISBSTIN(root)`

```
 1: procedure ISBSTIN(node)
 2:     if node = NULL then
 3:         return true
 4:     end if
 5:     leftOK <- ISBSTIN(node.left)
 6:     if leftOK = false then
 7:         return false
 8:     end if
 9:     if node.data <= prev then
10:         return false
11:     end if
12:     prev <- node.data
13:     return ISBSTIN(node.right)
14: end procedure
```

Line	Operation	Cost	Frequency	Count for S	Count for P
2	Test node = NULL	1	$2n+1$	15	5
3	Return true (NULL child)	1	$n+1$	8	2
5	Recursive call on left child	1	n	7	3
6	Test leftOK = false	1	n	7	3
7	Return false (left failed)	1	0	0	1
9	Test node.data $\leq$ prev	1	n	7	2
10	Return false (order broken)	1	0	0	1
12	prev $\leftarrow$ node.data	1	n	7	1
13	Recursive call on right child	1	n	7	1
	Total $T(n)$		$8n+2$	58	19

$T(n) = (2n+1) + (n+1) + 5n = 8n+2$, and $T(7) = 8(7)+2 = 58$, the same as the Count for S column.

Comparison: on a valid tree the range method costs $7n+2 = 51$ and the inorder method $8n+2 = 58$; both are $\Theta(n)$ in the worst case. The best cases differ. The range method checks a node the moment it is reached (preorder), so it can stop at the root's child: $\Theta(1)$, 8 steps on P. The inorder method cannot check anything until it has walked down to the leftmost node, so it must first go down the left edge of the tree: its best case is $\Theta(d)$, where d is the depth of the leftmost node, which is $\Theta(\log n)$ for a complete tree like P (19 steps) and $\Theta(1)$ only when the leftmost node is close to the root.

Question 15 (8 points): Vertical Order Traversal

(a) Pseudocode

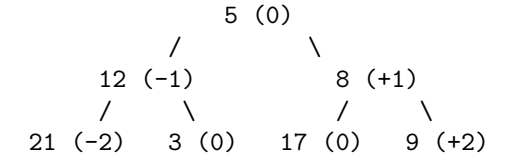
Level order is run with (node, distance) pairs in the queue, and each value is appended to the list of its column. Distances lie between $-(n-1)$ and $n-1$, so an array of $2n-1$ lists indexed by $d+n-1$ can hold every column. Because the queue gives level order, each column list is already top to bottom and left to right. minD and maxD record which columns are used, so only those are printed.

Algorithm 15: Vertical Order Traversal
Input: a pointer `root` to the root of a binary tree with n nodes.
Output: the values printed column by column from the leftmost distance to the rightmost.
Initial call: `VERTICAL(root, n)`

```
 1: procedure VERTICAL(root, n)
 2:     if root = NULL then
 3:         return
 4:     end if
 5:     col <- array of 2n - 1 empty lists
 6:     minD <- 0
 7:     maxD <- 0
 8:     Q <- empty queue
 9:     ENQUEUE(Q, (root, 0))
10:     while Q is not empty do
11:         (node, d) <- DEQUEUE(Q)
12:         APPEND(col[d + n - 1], node.data)
13:         minD <- MIN(minD, d)
14:         maxD <- MAX(maxD, d)
15:         if node.left is not NULL then
16:             ENQUEUE(Q, (node.left, d - 1))
17:         end if
18:         if node.right is not NULL then
19:             ENQUEUE(Q, (node.right, d + 1))
20:         end if
21:     end while
22:     for d <- minD to maxD do
23:         PRINT "distance ", d, ": "
24:         for each value v in col[d + n - 1] do
25:             PRINT v
26:         end for
27:     end for
28: end procedure
```

(b) Worked Example

Horizontal distances in the reference tree:



One row per pass of the while loop:

Pass	Dequeue (node, d)	Column after append	minD	maxD	Queue after (front first)
1	(5, 0)	d = 0: 5	0	0	(12, -1), (8, +1)
2	(12, -1)	d = -1: 12	-1	0	(8, +1), (21, -2), (3, 0)
3	(8, +1)	d = +1: 8	-1	1	(21, -2), (3, 0), (17, 0), (9, +2)
4	(21, -2)	d = -2: 21	-2	1	(3, 0), (17, 0), (9, +2)
5	(3, 0)	d = 0: 5, 3	-2	1	(17, 0), (9, +2)
6	(17, 0)	d = 0: 5, 3, 17	-2	1	(9, +2)
7	(9, +2)	d = +2: 9	-2	2	empty

Printing (lines 22 to 25), with minD $= -2$ and maxD $= +2$:

Distance d	Values printed	Line 25 runs	Line 24 tests
-2	21	1	2
-1	12	1	2
0	5, 3, 17	3	4
+1	8	1	2
+2	9	1	2

Final output: distance -2: 21 | -1: 12 | 0: 5, 3, 17 | $+1$: 8 | $+2$: 9, which matches the expected output.

(c) Frequency Count

C is the number of columns, $maxD - minD + 1$ (5 for the reference tree). Creating the array costs one step per empty list. As in Question 2, lines 16 and 19 add to $n - 1$.

Line	Operation	Cost	Frequency	Count for the example
2	Test root = NULL	1	1	1
3	Return (empty tree)	1	0	0
5	Create one empty list (per list)	1	$2n - 1$	13
6	minD $\leftarrow 0$	1	1	1
7	maxD $\leftarrow 0$	1	1	1
8	Create empty queue	1	1	1
9	Enqueue (root, 0)	1	1	1
10	Test Q is not empty	1	$n + 1$	8
11	Dequeue (node, d)	1	n	7
12	Append node.data to its column	1	n	7
13	Update minD	1	n	7
14	Update maxD	1	n	7
15	Test node.left is not NULL	1	n	7
16	Enqueue (left, d - 1)	1	$\lfloor n/2 \rfloor$	3
18	Test node.right is not NULL	1	n	7
19	Enqueue (right, d + 1)	1	$\lfloor (n-1)/2 \rfloor$	3
22	Test of the column loop	1	$C + 1$	6
23	Print column heading	1	C	5
24	Test of the value loop	1	$n + C$	12
25	Print one value	1	n	7
	Total $T(n)$		$12n + 3C + 5$	104

$T(n) = 1 + (2n-1) + 4 + (n+1) + 6n + (n-1) + (C+1) + C + (n+C) + n = 12n + 3C + 5$. Check: $T(7) = 12(7) + 3(5) + 5 = 104$, the same as the Count column. The grouping costs n appends plus $2n - 1$ steps to create the lists, and the printing costs $3C + 2n + 1$ steps.

(d) Final Complexity

Since $C \leq n$, the dominant term is $12n$ (and $T(n) \leq 15n + 5$). Best case $\Theta(n)$ and worst case $\Theta(n)$: every node is enqueued, grouped and printed once. The largest count, $15n + 5$, comes from a chain going only left (or only right), which has the most columns, $C = n$. Extra space is $\Theta(n)$: the $2n - 1$ column lists and the queue, which holds up to about $n/2$ pairs for a complete tree.

Twist

Each column holds at least one node and two nodes in different columns are different nodes, so a tree of n nodes has at most n columns. The maximum $C = n$ is reached by a chain that always goes left (distances $0, -1, \ldots, -(n-1)$) or always right. The minimum is 1 column for $n = 1$ and 2 columns for $n \geq 2$, for example a chain that goes left, right, left, right (distances $0, -1, 0, -1, \ldots$). The reference tree has $C = 5$ columns, from $-h$ to $+h$.

References Used For Learning Some Topics

1. GeeksforGeeks, "Tree" video collection.
   `https://ui.geeksforgeeks.org/videos/tag/tree/`
2. A. Bari, "1.4 Frequency Count Method," YouTube.
   `https://www.youtube.com/watch?v=1U3Uwct45IY`
3. E. Demaine, "Lecture 6: Binary Trees, Part 1," MIT 6.006 Introduction to Algorithms, Spring 2020, MIT OpenCourseWare.
   `https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/resources/lecture-6-binary-trees-part-1/`
4. S. Halim, VisuAlgo, "Binary Search Tree, AVL Tree" visualization.
   `https://visualgo.net/en/bst`

I used AI to:

- Assist me in turning my work into a LaTeX document.
- Analyze my written pseudocode for errors and accuracy.
- Teach/explain to me concepts that I was confused about or didn't know well.

I did NOT use AI to:

- Solve any tables, pseudocode, or problem solving tasks.